CS7CS2 – Optimisation for Machine Learning
Student Name: Stephen Komolafe
Student Number: 21336975

Week 4 Assignment

Q1.

The quadratic function:

$$f(x, y) = x^2 + 100y^2,$$

was implemented with an initial point of $(x_0, y_0) = (2, 2),$ and run for $T = 200$ iterations for each optimisation algorithm. The analytical gradient:

$$\nabla f(x, y) = (2x, 200y),$$

was coded explicitly to ensure exact derivative computation. The implementation is shown below:

```
def f(x):
    return x[0]**2 + 100*x[1]**2

def grad_f(x):
    return np.array([2*x[0], 200*x[1]])

x0 = np.array([2.0, 2.0])
T = 200
```

This function is strongly convex but ill-conditioned, since the curvature in the y-direction is 100 times larger than in the x-direction. This creates elongated elliptical level sets and makes it a useful test problem for comparing optimisation algorithms.

Part (I)

Polyak Algorithm:

The Polyak step size algorithm was implemented using the update rule:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

where

$$\alpha_k = \frac{f(x_k) - f^*}{||\nabla f(x_k)||^2},$$

Since the minimum occurs at (0,0), the optimal value was set to $f^* = 0$.

```
def polyak(x0):
    x = x0.copy()
    values = []
    f_star = 0  # minimum value

    for k in range(T):
        g = grad_f(x)
        alpha = (f(x) - f_star) / (np.dot(g, g) + 1e-8)
        x = x - alpha * g
        values.append(f(x))
    return values
```

In the code, the step size is computed at every iteration using the current function value and gradient norm via `np.dot(g,g)`. making the algorithm fully adaptive. The small constant, `1e-8`, was added to avoid division by zero. The function value at each iteration was stored in a list for plotting, allowing the convergence behaviour to be analysed.

RMSProp Algorithm:

RMSProp was implemented using an exponentially weighted average of squared gradients. The update takes the form

$$x_{k+1} = x_k - \alpha\left(\frac{\nabla f(x_k)}{\sqrt{(v_k)}+\varepsilon}\right),$$

where

$$v_k = \beta v_{k-1} + (1-\beta)\big(\nabla f(x_k)\big)^2.$$

```
def rmsprop(x0, alpha=0.2, beta=0.9, eps=1e-8):
    x = x0.copy()
    v = np.zeros_like(x)
    values = []

    for k in range(T):
        g = grad_f(x)
        v = beta*v + (1-beta)*(g**2)
        x = x - alpha * g / (np.sqrt(v) + eps)
        values.append(f(x))
    return values
```

In the code, v stores the running average of squared gradients, and the division by `np.sqrt(v) + eps` confirms coordinate-wise scaling of the step. This is necessary for this function since the gradient in the y-direction is much larger than in the x-direction. The parameters used were $\alpha = 0.2$ and $\beta = 0.9$.

Heavy Ball Algorithm:

The Heavy Ball algorithm introduces momentum through a velocity term. The implemented update is:

$$x_{k+1} = x_k + v_k$$

where

$$v_k = \beta v_{k-1} + \alpha\big(\nabla f(x_k)\big)$$

```
def heavy_ball(x0, alpha=0.01, beta=0.9):
    x = x0.copy()
    v = np.zeros_like(x)
    values = []
    for k in range(T):
        g = grad_f(x)
        v = beta*v - alpha*g
        x = x + v
        values.append(f(x))
    return values
```

Here, v accumulates past gradient information, which accelerates motion along shallow directions and reduces zig-zag oscillations. The parameters used were $\alpha = 0.01$ and $\beta = 0.9$.

Adam ($\alpha = 0.1,\ \beta_1 = 0.9, \beta_2 = 0.999$):

Adam combines momentum with adaptive scaling by maintaining both first and second moment estimates of the gradient. The moment updates are:

$$m_k = \beta_1 m_{k-1} + (1-\beta_1)\nabla f(x_k), \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2)\left(\nabla f(x_k)\right)^2,$$

followed by bias correction `m_hat` and `v_hat`:

$$\hat{m}_k = \frac{m_k}{(1-\beta_1^k)}, \qquad \hat{v}_k = \frac{v_k}{(1-\beta_2^k)}.$$

Altogether this gives the final update rule:

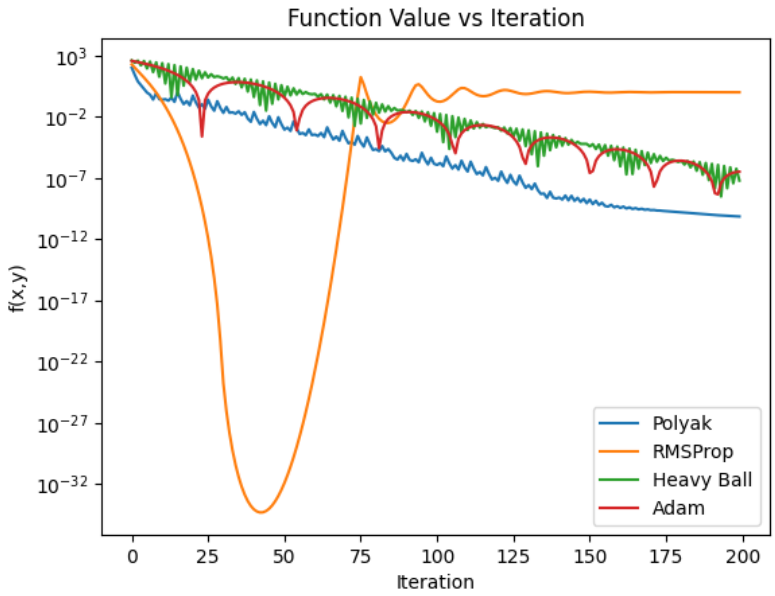
$$x_{k+1} = x_k - \alpha\left(\frac{\hat{m}_k}{\sqrt{(\hat{v}_k)} + \varepsilon}\right)$$

The parameters used were $\alpha = 0.1,\ \beta_1 = 0.9,$ and $\beta_2 = 0.999$.

```
def adam(x0, alpha=0.1, beta1=0.9, beta2=0.999, eps=1e-8):
    x = x0.copy()
    m = np.zeros_like(x)
    v = np.zeros_like(x)
    values = []

    for k in range(1, T+1):
        g = grad_f(x)
        m = beta1*m + (1-beta1)*g
        v = beta2*v + (1-beta2)*(g**2)
        m_hat = m / (1 - beta1**k)
        v_hat = v / (1 - beta2**k)
        x = x - alpha*m_hat / (np.sqrt(v_hat) + eps)
        values.append(f(x))
    return values
```

The function values from each algorithm were plotted on the same graph using a logarithmic scale on the vertical axis to clearly compare convergence rates across several orders of magnitude.

```
polyak_vals = polyak(x0)
rms_vals = rmsprop(x0)
hb_vals = heavy_ball(x0)
adam_vals = adam(x0)

plt.figure()
plt.plot(polyak_vals, label="Polyak")
plt.plot(rms_vals, label="RMSProp")
plt.plot(hb_vals, label="Heavy Ball")
plt.plot(adam_vals, label="Adam")
plt.yscale("log")
plt.xlabel("Iteration")
plt.ylabel("f(x,y)")
plt.legend()
plt.title("Function Value vs Iteration")
plt.show()
```



The plot shows that all algorithms converge toward the global minimum f^*, but their convergence behaviour differs significantly. The Polyak step size produces a smooth and monotonic decrease due to its adaptive scaling based on the objective value. The Heavy Ball algorithm decreases more rapidly in early iterations because momentum accelerates movement along the shallow x-direction, although mild oscillations are visible before the algorithm stabilises. RMSProp substantially reduces oscillations by shrinking updates in the high-curvature y-direction, resulting in more stable progress. Adam exhibits both fast initial convergence and smooth decay, as it combines momentum with coordinate-wise adaptive normalisation. The adaptive algorithms handle the ill-conditioning of the quadratic function more successfully, but Heavy Ball accelerates convergence at the expense of possibly oscillatory behaviour depending on parameter choice.

Part (II)

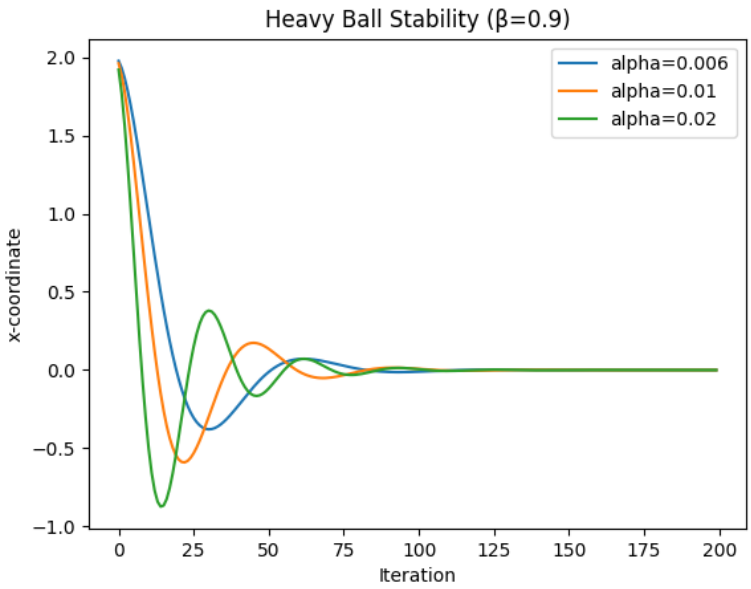
To investigate the stability of the heavy ball method, the momentum parameter was fixed at $\beta = 0.9$ while the step size α was gradually increased. The function `heavy_ball_track` implements the same heavy ball update rule used in Part (I), but instead of storing the function value, it records only the x-coordinate at each iteration. This allows the behaviour of the method to be examined more clearly along a single direction as the step size varies.

```
def heavy_ball_track(x0, alpha, beta=0.9):
    x = x0.copy()
    v = np.zeros_like(x)
    x_values = []
    for k in range(T):
        g = grad_f(x)
        v = beta*v - alpha*g
        x = x + v
        x_values.append(x[0])

    return x_values
```

In the code implementation, v is initialised as a zero vector and updated at every iteration using the current gradient. The update follows the same momentum structure as before, but only `x[0]` is appended to the list `x_values`. This isolates the evolution of the x-component and makes oscillatory behaviour easier to observe while varying the step size.

```
alphas = [0.006, 0.01, 0.02]

plt.figure()
for a in alphas:
    x_vals = heavy_ball_track(x0, a)
    plt.plot(x_vals, label=f"alpha={a}")

plt.xlabel("Iteration")
plt.ylabel("x-coordinate")
plt.legend()
plt.title("Heavy Ball Stability (β=0.9)")
plt.show()
```

The step sizes, $\alpha = 0.006, 0.01, 0.02$, were tested. For each value of α, the function `heavy_ball_track(x0,a)` was called, and the resulting x-coordinate sequence was plotted against iteration number.



The resulting stability plot clearly demonstrates the effect of increasing the step size. When $\alpha = 0.006$, the method converges stably, although relatively slowly, with small oscillations that gradually decay. Increasing the step size to $\alpha = 0.01$ produces faster convergence while remaining stable; mild oscillations appear initially due to the momentum term but are damped over time. However, when $\alpha = 0.02$, the iterates become unstable, and the oscillations grow in magnitude rather than diminishing, indicating that the step size exceeds the stability limit of the algorithm.

This behaviour is consistent with the ill-conditioned nature of the quadratic function. Because the curvature in the y-direction is much larger than in the x-direction, the allowable step size is constrained by the steepest direction. As α increases, the momentum term amplifies oscillations instead of damping them, eventually leading to

divergence. This demonstrates the sensitivity of the heavy ball method to step size selection and shows how momentum can both accelerate convergence and induce instability when parameters are not carefully chosen.

Part (III)

To visualise the geometric behaviour of the different optimisation algorithms, contour lines of the quadratic function were plotted, and the optimisation trajectories were overlaid. The contour lines were generated by constructing a grid of (x, y) values and computing:

$$Z = X^2 + 100Y^2$$

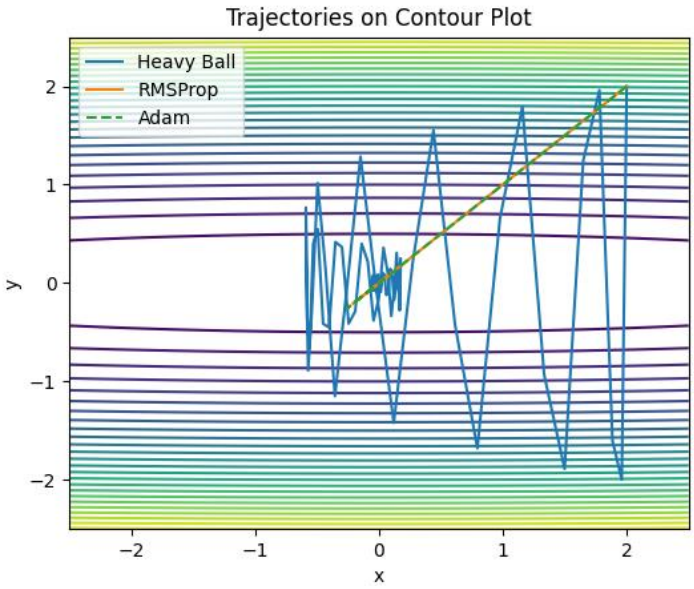
which produces elongated elliptical level sets. These ellipses reflect the strong curvature imbalance between the two directions: the curvature in the y-direction is much larger than in the x-direction, creating a narrow valley aligned with the x-axis.

The function `trajectory(algo)` was written to compute and store the full sequence of iterates for each optimisation method.

```
def trajectory(algo):
    x = x0.copy()
    traj = [x.copy()]

    if algo == "hb":
        v = np.zeros_like(x)
        alpha=0.01; beta=0.9
        for _ in range(T):
            g = grad_f(x)
            v = beta*v - alpha*g
            x = x + v
            traj.append(x.copy())

    elif algo == "rms":
        v = np.zeros_like(x)
        alpha=0.2; beta=0.9; eps=1e-8
        for _ in range(T):
            g = grad_f(x)
            v = beta*v + (1-beta)*(g**2)
            x = x - alpha*g/(np.sqrt(v)+eps)
            traj.append(x.copy())

    elif algo == "adam":
        m = np.zeros_like(x)
        v = np.zeros_like(x)
        alpha=0.1; b1=0.9; b2=0.999; eps=1e-8
        for k in range(1,T+1):
            g = grad_f(x)
            m = b1*m + (1-b1)*g
            v = b2*v + (1-b2)*(g**2)
            m_hat = m/(1-b1**k)
            v_hat = v/(1-b2**k)
            x = x - alpha*m_hat/(np.sqrt(v_hat)+eps)
            traj.append(x.copy())

    return np.array(traj)
```

This function computes the sequence of iterates starting from the same initial point. In each case, the current iterate `x.copy()` the is appended to the list `traj`, and the final trajectory is returned as a NumPy array for plotting. The heavy ball method uses the

same velocity-based update as before with $\alpha = 0.01$ and $\beta = 0.9$. RMSProp maintains an exponential moving average of squared gradients and applies coordinate-wise scaling to the update. Adam computes both first and second moment estimates, applies bias correction, and then performs an adaptively scaled update.



The above contour plot reveals the geometric differences in how each algorithm navigates the narrow valley. Heavy Ball follows a characteristic zig-zag path across the valley walls. Although momentum accelerates movement along the shallow x-direction and reduces some oscillations compared to plain gradient descent, overshooting still occurs due to the steep curvature in the steep y-direction. As a result, the trajectory alternates across the valley before gradually settling toward the minimum.

RMSProp produces a more direct path toward the origin. Because it scales updates inversely proportional to the accumulated squared gradients, the large gradients in the y-direction are automatically damped. This coordinate-wise normalisation reduces oscillations and allows smoother progress within the valley.

Adam exhibits the most balanced behaviour among the three methods. By combining momentum with adaptive scaling, it moves efficiently along the shallow direction while simultaneously controlling oscillations in the steep direction. Its trajectory appears smoother and more centrally aligned within the valley, reflecting improved handling of the function's ill-conditioning.

The zig-zag behaviour observed in Heavy Ball is caused by the high curvature difference between the x and y coordinates. Since the gradient in the y-direction is much larger, updates tend to overshoot across the valley walls. Adaptive methods effectively compensate for this curvature imbalance, producing more stable and direct convergence paths toward the global minimum.

Q2.

The Rosenbrock function:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2,$$

was implemented together with its partial derivative gradients `dfdx` and `dfdy` to ensure exact derivative calculations instead of relying on numerical approximation. The partial derivatives were derived explicitly as:

$$\frac{df}{dx} = -2(1 - x) - 400x(y - x^2), \qquad \frac{df}{dy} = 200(y - x^2).$$

These expressions were coded directly as follows:

```
def f(x):
    return (1 - x[0])**2 + 100*(x[1] - x[0]**2)**2

def grad_f(x):
    dfdx = -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
    dfdy = 200*(x[1] - x[0]**2)
    return np.array([dfdx, dfdy])

x0 = np.array([-1.25, 0.5])
T = 3000
```

The initial point was set to $(x_0, y_0) = (-1.25, 0.5)$, which lies outside the curved valley of the Rosenbrock function, making the optimisation problem more challenging. The number of iterations was increased to $T = 3000$ to allow sufficient time for convergence, as this function is significantly harder to optimise than the quadratic function in **Q1**.

Part (I)

The same optimisation algorithms and update rules from **Q1** were applied here, using the specified parameter values.

Polyak Algorithm:

For the Polyak implementation, however, due to the Rosenbrock function being non-convex and highly curved, the raw Polyak step size can become excessively large when far from the minimum. To prevent instability, the computed step was truncated using a maximum allowable value of `alpha_max=0.1`.

```
def polyak(x0, alpha_max=0.1):
    x = x0.copy()
    values = []
    f_star = 0

    for k in range(T):
        g = grad_f(x)
        alpha = (f(x) - f_star) /
(np.dot(g,g) + 1e-8)
        alpha = min(alpha, alpha_max)
        x = x - alpha * g
        values.append(f(x))
    return values
```

RMSProp Algorithm:

RMSProp was implemented using $\alpha = 0.1$ and $\beta = 0.9$. As in **Q1**, it maintains an exponential moving average of squared gradients to adaptively scale each coordinate of the update:

```
def rmsprop(x0, alpha=0.01, beta=0.9,
eps=1e-8):
    x = x0.copy()
    v = np.zeros_like(x)
    values = []

    for k in range(T):
        g = grad_f(x)
        v = beta*v + (1-beta)*(g**2)
        x = x - alpha*g/(np.sqrt(v)+eps)
        values.append(f(x))
    return values
```

Heavy Ball Algorithm:

Heavy Ball was implemented with a much smaller learning rate $\alpha = 2 \times 10^{-4}$ and momentum parameter $\beta = 0.9$. The small step size is necessary as the curvature of the Rosenbrock function varies significantly along and across the valley:

```
def heavy_ball(x0, alpha=2e-4, beta=0.9):
    x = x0.copy()
    v = np.zeros_like(x)
    values = []

    for k in range(T):
        g = grad_f(x)
        v = beta*v - alpha*g
        x = x + v
        values.append(f(x))
    return values
```

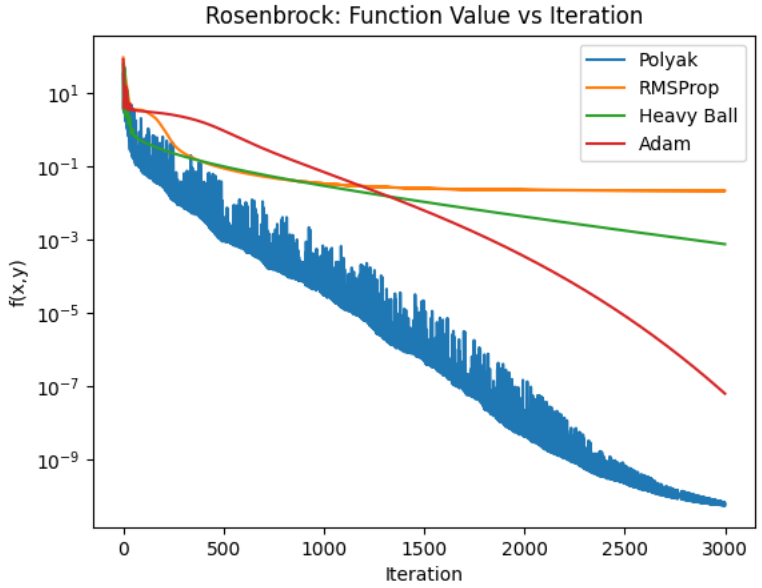
Adam Algorithm:

Adam was implemented with $\alpha = 0.05$, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. As before, it maintains the first and second moment estimates with bias correction before applying the update:

```
def adam(x0, alpha=0.05, beta1=0.9,
beta2=0.999, eps=1e-8):
    x = x0.copy()
    m = np.zeros_like(x)
    v = np.zeros_like(x)
    values = []

    for k in range(1, T+1):
        g = grad_f(x)
        m = beta1*m + (1-beta1)*g
        v = beta2*v + (1-beta2)*(g**2)

        m_hat = m/(1-beta1**k)
        v_hat = v/(1-beta2**k)
```

```
        x = x -
alpha*m_hat/(np.sqrt(v_hat)+eps)
        values.append(f(x))
    return values
```

Each algorithm stores the function value at every iteration, and the four sequences were plotted together on a logarithmic scale to compare convergence behaviour over several orders of magnitude.



The convergence behaviour is significantly different from the quadratic case in **Q1**. The Polyak step size decreases the objective function but does so more slowly and less smoothly than before. Since the Rosenbrock function is non-convex and features a curved valley, the Polyak formula no longer provides near-optimal step sizes throughout the trajectory, and truncation becomes necessary for stability.

Heavy ball initially makes noticeable progress but exhibits oscillations as it enters the narrow-curved valley. The very small learning rate 2×10^{-4} is required to prevent divergence due to the high curvature perpendicular to the valley floor. Even with momentum, convergence along the valley remains slow.

RMSProp demonstrates more stable descent than heavy ball. By scaling updates coordinate-wise according to accumulated squared gradients, it reduces the effect of steep curvature near the valley walls and produces smoother progress.

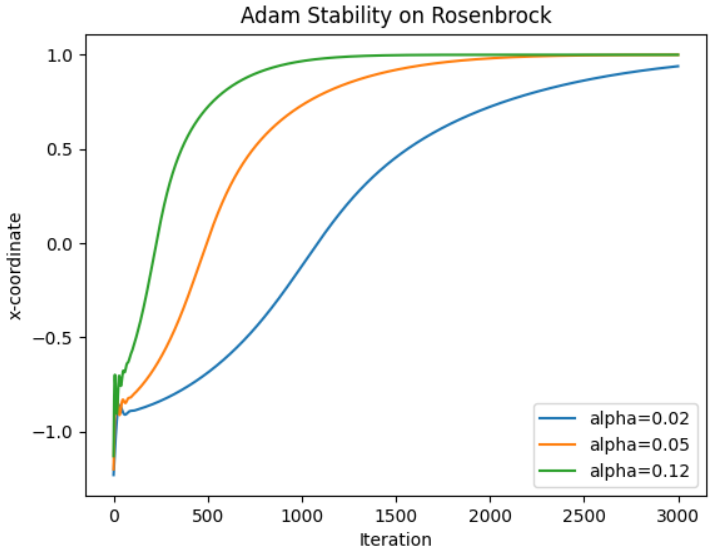
Adam achieves the fastest and most stable reduction in function value among the four methods. The combination of momentum and adaptive scaling allows it to align more effectively with the curved valley structure. However, convergence remains significantly slower than in **Q1** due to movement along the valley floor being controlled by very small gradients, while the perpendicular direction exhibits large curvature. This imbalance in curvature limits the achievable step size resulting in slower convergence.

Part (II)

The stability of Adam on the Rosenbrock function was examined by fixing the algorithmic structure and varying only the learning rate α. Rather than storing function values, the x-coordinate of the iterate, `x[0]`, was recorded at each step. Tracking a single coordinate makes oscillatory behaviour and divergence more visually apparent, particularly as the iterates move through the narrow-curved valley. The learning rates $\alpha = 0.02,\ 0.05,$ and 0.12 were tested.

```
def adam_track(x0, alpha):
    x = x0.copy()
    m = np.zeros_like(x)
    v = np.zeros_like(x)
    x_vals = []
    beta1 = 0.9
    beta2 = 0.999
    eps = 1e-8

    for k in range(1, T+1):
        g = grad_f(x)
        m = beta1*m + (1-beta1)*g
        v = beta2*v + (1-beta2)*(g**2)
        m_hat = m/(1-beta1**k)
        v_hat = v/(1-beta2**k)
        x = x - alpha*m_hat/(np.sqrt(v_hat)+eps)
        x_vals.append(x[0])

    return x_vals

alphas = [0.02, 0.05, 0.12]
```

The following plot was generated from the functions returned values:



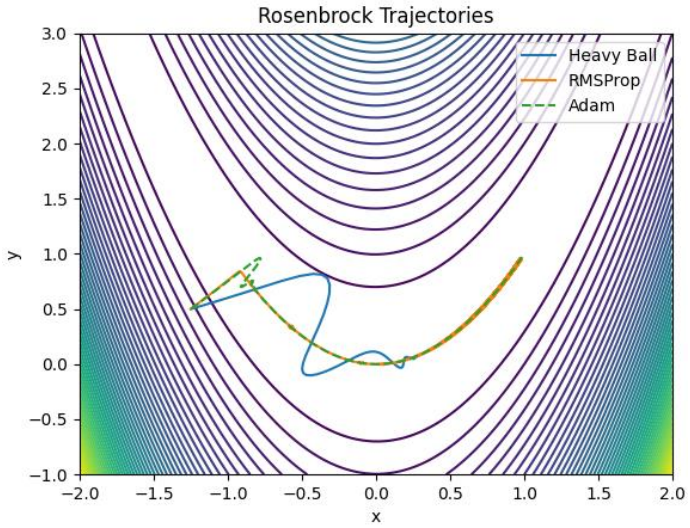
The resulting stability plot shows clear differences as the learning rate increases. When $\alpha = 0.02$, the method converges stably, although progress along the curved valley is relatively slow. Increasing the learning rate to $\alpha = 0.05$ produces faster convergence while remaining stable; small oscillations are present in early iterations but are well controlled by the adaptive scaling and momentum terms. However, when $\alpha = 0.12$, the method becomes unstable. The iterates exhibit large oscillations and may move away from the valley instead of settling toward the minimiser.

This demonstrates that even adaptive algorithms like Adam have a stability threshold that depends on the curvature of the objective function. The Rosenbrock function contains regions of very high curvature perpendicular to the valley and much lower curvature along the valley floor. When the step size becomes too large, the updates overshoot across the narrow valley walls, and the stabilising effect of adaptive scaling is no longer sufficient to prevent divergence.

Part (III)

To further understand the geometric behaviour of the algorithms, contour lines of the Rosenbrock function were plotted over a grid. These contours reveal the characteristic banana-shaped valley, within which the global minimum at $(1, 1)$ lies. The full optimisation trajectories of Heavy Ball, RMSProp, and Adam were then overlaid on this contour plot using the parameter values fixed in ***(I)***.

```
def trajectory(algo):
    x = x0.copy()
    traj = [x.copy()]

    if algo == "hb":
        v = np.zeros_like(x)
        for _ in range(T):
            g = grad_f(x)
            v = 0.9*v - 2e-4*g
            x = x + v
            traj.append(x.copy())

    elif algo == "rms":
        v = np.zeros_like(x)
        for _ in range(T):
            g = grad_f(x)
            v = 0.9*v + 0.1*(g**2)
            x = x - 0.01*g/(np.sqrt(v)+1e-8)
            traj.append(x.copy())

    elif algo == "adam":
        m = np.zeros_like(x)
        v = np.zeros_like(x)
        for k in range(1,T+1):
            g = grad_f(x)
            m = 0.9*m + 0.1*g
            v = 0.999*v + 0.001*(g**2)
            m_hat = m/(1-0.9**k)
            v_hat = v/(1-0.999**k)
            x = x - 0.05*m_hat/(np.sqrt(v_hat)+1e-8)
            traj.append(x.copy())

    return np.array(traj)
```



Heavy Ball initially moves rapidly toward the valley but then oscillates across it. While momentum accelerates motion along flatter directions, it can also amplify overshooting when the valley curves, leading to visible zig-zag behaviour.

RMSProp follows a smoother path inside the valley. By scaling updates according to accumulated squared gradients, it reduces aggressive motion in steep directions and adapts more effectively to local curvature changes.

Adam exhibits a balanced trajectory akin to that of RMSProp. The momentum component sustains progress along the valley floor, while adaptive normalisation stabilises updates as the path bends through curved regions.

Rosenbrock's Slow Convergence:

The slow convergence of the Rosenbrock function arises from its narrow, curved valley structure. Gradients perpendicular to the valley are large, causing oscillations and potential overshoot, whereas gradients along the valley floor are small, slowing forward progress.

Momentum reduces the zig-zag motion but may increase overshoot if the step size is too large. Adaptive methods improve stability by scaling updates relative to gradient magnitude, effectively compensating for differences in curvature across coordinates. However, because curvature varies significantly across both direction and location, the allowable stability limit for learning rate remains sensitive. Larger step sizes amplify oscillations, particularly near the steep valley walls, highlighting the difficulty of this optimisation landscape.